

Verifica di qualità, architettura e sicurezza

Summer Booking – frontend Angular 22 (`summer-booking-front-office`), stato del 06/10/2026, branch `dev` con le modifiche non ancora committate. La verifica legge il codice e la documentazione del repository; non ha provato il backend reale né la configurazione AWS di produzione. Nel documento non compaiono credenziali né dati personali.

1. In sintesi

Nessuna violazione dei livelli dell'architettura	273 / 273 test superati; lint e build puliti	3 alte criticità di sicurezza, tutte nel contratto API o nel backend	7 medie derivate di qualità nel frontend, nessuna bloccante
--	--	--	---

Siamo andati fuori strada? Sull'architettura no: i livelli (entità, servizi, behaviours, viste, wrapper) sono rispettati, i wrapper restano pronti per diventare una libreria e i mock non entrano nella build di produzione. Ci sono invece **derivate di coerenza**: alcune regole sono nate strada facendo (validazioni con i Signal Forms, errori sotto il campo, distruzione esplicita) e le pagine costruite prima non sono ancora allineate. Si sistemano con poco lavoro, se si fa adesso, prima che il modello si copi in altre pagine.

Sicurezza: il frontend segue bene le indicazioni OWASP per una SPA. Il token è solo in memoria, il refresh è nel cookie `HttpOnly` e il Bearer va solo alla nostra API. Non ci sono HTML non sanificati né segreti nel codice. Le criticità vere stanno nel **contratto della configurazione email** (password SMTP restituita in chiaro, test email che apre connessioni verso host scelti dall'utente) e nei **controlli che solo il backend può fare** (lo stabilimento `idProperty` arriva dal client). Restano poi alcune configurazioni dell'infrastruttura da completare (CSP, header in produzione).

2. Cosa è cambiato oggi (richiesta sulla porta SMTP)

- **Number field con i pallini:** `app-filled-number-field` ha il nuovo parametro `masked`, con l'occhietto in fondo come i campi password (icone `[valuta, contenuto visibile, contenuto nascosto]`). Si aggiungono `labelledBy`, `autocomplete` e l'etichetta facoltativa. La porta SMTP ora usa questo campo: si possono scrivere solo cifre.
- **Validazione reattiva della porta:** è obbligatoria e va da 1 a 65535 (`required`, `min`, `max` dei Signal Forms). Il messaggio compare sotto il campo ("Questo campo è obbligatorio", "Il valore minimo è 1", "Il valore massimo è 65535") e **Salva resta spento** finché c'è un errore.
- **Errori del backend sotto il campo che li genera:** gli invii passano da `submit()` dei Signal Forms. Gli `errors[]` del backend vanno sotto il campo con lo stesso nome (`apiFieldErrors`) e spariscono appena il campo cambia. Anche la tendina `app-filled-select` ora mostra un errore. Sotto il riquadro resta solo l'errore che non riguarda un campo, per esempio la rete assente.
- **Limite dei Signal Forms:** le regole non controllano i campi in sola lettura. Prima di "Sì", quindi, un errore del backend su un campo compare sotto il riquadro.
- **Già corretti nel codice nuovo:** tre punti segnalati dalla verifica (`takeUntilDestroyed` vietato dalle regole, testo di `error.unknown` duplicato, commento superfluo).

3. Sicurezza (riferimento OWASP Top 10 e ASVS)

Legenda "Chi": FE = frontend, BE = backend, Infra = AWS/CloudFront. Il comportamento del backend non è nel repository: dove serve, il punto è scritto come requisito da verificare.

#	Gravità	Problema	Correzione	Chi
S1	ALTA A04 / A02 ASVS V8	Password SMTP restituita in chiaro. <code>GET email-configuration</code> restituisce <code>smtpPassword</code> : il segreto arriva nel browser, resta in memoria e nel campo, e con l'occhietto lo legge chiunque usi la sessione. Salva e Invia email test lo rimandano sempre, anche se non è cambiato.	Password "solo scrittura": - la GET non la restituisce mai (al massimo <code>hasSmtpPassword: true</code>); - il campo parte vuoto con "imposta"; - la richiesta la manda solo se cambiata (<code>null</code> = invariata) e il test usa quella salvata; - il backend la cifra (KMS / Secrets Manager) e risponde con <code>Cache-Control: no-store</code>. Va aggiornato API_CONTRACTS.	BE + FE
S2	ALTA A10 SSRF	Invia email test fa aprire al server connessioni verso host e porta scelti dall'utente. Il server può essere usato per sondare la rete interna (anche i metadata AWS 169.254.169.254) o per mandare spam.	Lato server: - bloccare gli IP privati, di loopback e link-local, controllando l'indirizzo dopo la risoluzione DNS; - porte ammesse solo 25, 465 e 587; - timeout brevi e limite di frequenza; - destinatario fisso, cioè l'utente autenticato; - messaggio d'errore generico, senza il dettaglio SMTP.	BE
S3	ALTA A01 Broken Access Control	idProperty scelto dal client in tutte le API (magazzino, impostazioni, assistenza). <code>authGuard</code> protegge solo la navigazione nel browser. Il rischio è un IDOR: dati di un altro stabilimento letti o modificati.	Il backend controlla a ogni chiamata che l'utente del token appartenga a quello stabilimento con il ruolo adatto (per la configurazione email, un amministratore); in mancanza del controllo, rifiuta. Non verificabile dal frontend.	BE
S4	MEDIA A05	CSP assente. La build di produzione inserisce in <code>index.html</code> uno <code><style></code> e un <code>onload</code> inline, che una CSP severa bloccherebbe.	Già pianificata in <code>SECURITY_TODO.md</code> : <code>inlineCritical: false</code> (oppure <code>autoCsp</code>) in <code>angular.json</code>; - CSP dalla response headers policy di CloudFront, prima in modalità Report-Only; - aggiungere <code>Permissions-Policy</code> e <code>Cross-Origin-Opener-Policy</code>.	Infra + FE
S5	MEDIA A05	Header di sicurezza (HSTS, nosniff, X-Frame-Options, Referrer-Policy) documentati solo per DEV; la policy di PROD è "da creare". Nel repository non trovo come <code>/api/*</code> raggiunge il backend in CloudFront.	- policy degli header anche in PROD; - un comportamento <code>/api/*</code> dedicato, senza la funzione SPA, senza cache, che inoltri cookie e <code>Authorization</code>.	Infra
S6	MEDIA A01 (CSRF)	Refresh e logout usano il cookie: sono POST senza corpo e senza header personalizzati, quindi la difesa contro la CSRF deve stare sul server.	Lato server: - cookie <code>HttpOnly</code>; <code>Secure</code>; <code>SameSite=Strict</code>; <code>Path=/api/auth</code>, con il prefisso <code>__Host-</code> se possibile; - controllo di <code>Origin</code> / <code>Sec-Fetch-Site</code>; - CORS chiuso. Facoltativo: un header obbligatorio aggiunto dal frontend.	BE

#	Gravità	Problema	Correzione	Chi
S7	MEDIA A07	Account di test dei mock: l'username è un indirizzo di posta reale e l'hash SHA-256 della password è senza sale. Sono nel codice, nella cronologia Git e nella build DEV pubblica; il refresh simulato si aggira scrivendo a mano lo storage. Già registrato in <code>SECURITY_TODO.md</code> .	- far accettare al mock qualunque password per l'username di test, togliendo l'hash; - cambiare quella password ovunque sia usata; - limitare l'accesso a DEV (per IP o con autenticazione). La build di produzione non contiene i mock (verificato).	FE + Infra
S8	BASSA A06	<code>npm audit</code> : 1 vulnerabilità "high" in <code>@angular/router</code> 22.1.7 (GHSA-ff3f-86qr-9cv3, DoS solo con il rendering lato server, che l'app non usa). Inoltre <code>core</code> , <code>common</code> , <code>router</code> e <code>forms</code> sono a 22.1.7, mentre <code>material</code> e <code>cdk</code> sono a 22.2.0.	Portare tutti i pacchetti <code>@angular/*</code> alla 22.2.x insieme, con <code>ng update</code> .	FE
S9	BASSA A07	Nessun limite ai tentativi su accesso, contatto assistenza e test email, e nessun codice d'errore per il 429.	Limite di frequenza e blocco progressivo nel backend; un codice (es. <code>auth.too_many_attempts</code>) da tradurre nel frontend.	BE + FE
S10	BASSA A07	Rinnovo del token con più schede aperte: due schede possono rinnovare insieme con lo stesso cookie. Dentro una scheda le richieste sono già unite.	Se il backend rileva il riuso del refresh token, una breve finestra di tolleranza evita logout immotivati.	BE
S11	BASSA A03	Modulo Assistenza e guide: nel frontend solo <code>required</code> , senza lunghezza massima né formato email.	Lato server: - validare lunghezze e formati; - togliere CR/LF da ciò che va negli header della mail (header injection); - mail in testo semplice; - mai l'email dell'utente come mittente. Lato frontend: <code>maxLength</code> ed <code>email()</code> dei Signal Forms, solo per l'esperienza d'uso.	BE + FE
S12	BASSA	<code>autocomplete="new-password"</code> su tutti e sei i campi nascosti: i password manager possono proporre di generare o salvare una "password" anche su email, server e porta.	<code>new-password</code> solo sulla password; sugli altri <code>off</code> .	FE
S13	INFO	Il segnaposto del campo email del form di Assistenza è un indirizzo Gmail dall'aspetto reale (<code>it.json</code> e <code>en.json</code> , riga 284).	Usare un dominio riservato agli esempi (<code>@example.com</code> , RFC 2606).	FE
S14	INFO A03 XSS	L'unico <code>[innerHTML]</code> è in <code>app-i18n-text</code> , attivo solo con <code>[html]="true"</code> , usato una volta con traduzioni statiche; Angular lo sanifica.	Regola da mantenere: mai <code>[html]="true"</code> con parametri che arrivano dal server.	FE

Cosa è già fatto bene (verificato nel codice)

- Token e sessione:**
 - l'access token è solo in memoria e il refresh token nel cookie HttpOnly;
 - nessun token in storage, URL o console;
 - il logout revoca prima sul server, poi pulisce e avvisa le altre schede;
 - sul 401 c'è un solo rinnovo.
- Interceptor:** il Bearer va solo agli indirizzi della nostra API (`/api/`, stessa origine). URL come `//evil.com/api` o `/apix` non vengono riconosciuti come nostri.
- Mock esclusi dalla produzione:** lo fa `fileReplacements`, verificato anche nel bundle.
- Accesso:** nessun open redirect (nessun `returnUrl`).
- Errori:** mostrati sempre come testo, mai come HTML; `title`, `type` e `traceId` non compaiono all'utente.
- Risorse:** nessuno script o font da CDN; nessun dato del server reso come HTML.

4. Qualità del codice e architettura

Verificato e in regola:

- nessun import contrario ai livelli: i wrapper non importano behaviours, services, entità o ngx-translate;
- le entità non dipendono da Angular;
- nessun `!important`; ogni `!ng-deep` ha il commento che lo giustifica;
- gli SCSS sopra le 150 righe (header, home) sono già nel documento dei refactoring;
- lint, type-check e build sono puliti e i 273 test passano.

#	Gravità	Problema	Correzione proposta
Q1	MEDIA	takeUntilDestroyed vietato dalle nostre regole (la distruzione è sempre esplicita, e le chiamate HTTP finiscono da sole) è usato in 5 file: <code>contact-support</code> e i 4 popup del magazzino. Nella Configurazione email l'ho tolto oggi.	Togliere il <code>pipe(takeUntilDestroyed(...))</code> sulle chiamate HTTP. Se un caso deve davvero essere annullato, tenere la <code>Subscription</code> e chiamare <code>unsubscribe()</code> in <code>ngOnDestroy</code> .
Q2	MEDIA	Gestione degli errori ripetuta 7 volte (login, assistenza, configurazione email, 4 popup): • <code>signal<ApiProblem> + toSignal(toObservable(...).pipe(switchMap(errorText.text))) + toApiProblem;</code> • viste e componenti importano un helper dell'infrastruttura.	• Un metodo <code>ErrorTextBehaviour.message(signal)</code> che restituisce il testo come signal, come fa <code>ValidationTextBehaviour</code>. • Un interceptor che trasforma ogni errore in <code>ApiProblem</code> prima che arrivi alle pagine.
Q3	MEDIA	Popup Elimina e Duplica identici: cambiano solo i nomi (verificato, il confronto è vuoto dopo aver sostituito "delete"/"duplicate"). Non sono tra i dopponi accettati.	Un solo componente con l'azione come parametro; oppure, se li vuoi tenere separati come Aggiungi/Modifica, aggiungerli all'elenco dei dopponi accettati.
Q4	MEDIA	Popup "Aggiungi articolo" senza Signal Forms: usa segnali separati e un <code>canAdd</code> scritto a mano. La quantità 0 non mostra nessun messaggio, mentre il popup gemello "Modifica" usa già Signal Forms e <code>[error]</code> .	Portarlo ai Signal Forms (<code>required</code> sull'articolo, <code>greaterThan</code> sulla quantità), con i messaggi sotto i campi.
Q5	MEDIA	La nuova regola "errori sotto il campo" vale per ora solo nella Configurazione email. Assistenza e popup mostrano gli <code>errors[]</code> del backend come messaggio generico.	Applicare lo stesso modello (<code>submit()</code> + <code>apiFieldErrors</code>) ad Assistenza e ai popup con form.
Q6	MEDIA	Caricamenti fatti in modi diversi: • la Configurazione email non ricarica al cambio lingua (<code>DATA_RELOAD</code>, che la regola chiede per ogni caricamento); • l'elenco articoli del popup Aggiungi non dipende da <code>DATA_RELOAD</code>, non annulla una richiesta precedente e, se fallisce, mostra un elenco vuoto senza messaggio.	• Un solo modello per i caricamenti (<code>rxResource</code> con <code>idProperty</code> e <code>DATA_RELOAD</code> come parametri) e per gli invii (<code>submit()</code>). • Per la Configurazione email decidere se il cambio lingua deve ricaricare, perché perderebbe le modifiche; altrimenti scriverlo come eccezione.
Q7	MEDIA	LoginDialog riscrive apertura e chiusura invece di estendere <code>BasePopup</code> .	Estendere <code>BasePopup</code> e aggiungere due hook protetti per le sue esigenze (seguire lo schermo, azzerare il form).

#	Gravità	Problema	Correzione proposta
Q8	BASSA	Colori scritti a mano: #dc3545 in 3 SCSS (login, popup Aggiungi e Modifica) è già <code>--color__layout__error</code>; #365e96 (menu gestionale e menu impostazioni) non è nel tema; #ebebeb (icon-button) coincide quasi con <code>--color__tertiary__theme</code>; - il grigio #767676 per il contrasto (card, home) e l'ombra <code>rgb(18 45 87)</code> si ripetono.	Usare le variabili del tema e aggiungere al tema i colori condivisi mancanti.
Q9	BASSA	Test mancanti: - pagina Impostazioni magazzino (coordina 4 popup e la selezione); <code>ValidationTextBehaviour</code> e <code>apiFieldErrors</code>, provati solo attraverso le pagine; - header dell'hub, guard della lingua, alcuni wrapper minori. Un test di ambiente intermittente (<code>mql.addListener</code> in <code>jsdom</code>) a volte segnala errori non gestiti.	Aggiungere le spec, a partire dalla pagina del magazzino; fornire un <code>matchMedia</code> finto nel setup dei test.
Q10	BASSA	Accessibilità di <code>app-filled-select</code>: - senza etichetta e senza <code>labelledBy</code>, <code>aria-labelledby</code> punta a un id inesistente; - in sola lettura diventa disattivata, quindi esce dall'ordine del tasto Tab.	- Mettere <code>aria-labelledby</code> solo se c'è un nome. - In sola lettura: <code>aria-readonly</code> e blocco dell'apertura, senza disattivarla.
Q11	BASSA	Dettagli: <code>afterDuplicate</code> capisce "riga o selezione" da <code>itemName === null</code>, un legame implicito; <code>auth.user()?.idProperty</code> è ripetuto in 9 punti.	- Un segnale esplicito per l'origine della duplicazione. - Un <code>AuthBehaviour.idProperty</code> computed.
Q12	INFO	Per un campo di testo vuoto il backend (mock) risponde <code>validation.invalid_value</code> , cioè "Il valore scelto non è valido.", che suona strano su un campo da scrivere.	Concordare con il backend un codice <code>validation.required</code> per i campi obbligatori.

Punti forti

- Livelli CLEAN rispettati con disciplina; dipendenze solo verso l'interno.
- Angular moderno e coerente:** componenti standalone, OnPush e segnali ovunque, `effect` con `untracked` dove serve.
- Wrapper quasi pronti per la libreria:**
 - icone in ordine di posizione e testi tradotti dal chiamante;
 - colori come variabili CSS;
 - ARIA curato (`aria-invalid`, `aria-describedby`, `aria-pressed`).
- Errori con un flusso unico:** Problem Details → `ApiProblem` → chiave `error.<code>`, con un test che controlla le traduzioni.
- `BasePopup`, `SETTINGS_PAGES` e i mixin `settings-panel` / `table-page` evitano duplicazioni; i commenti spiegano quasi sempre il perché.

5. Priorità consigliate

- Contratto della configurazione email:** password solo in scrittura (S1) e protezione SSRF del test email (S2). Va concordato con il backend prima che lo implementi.
- Requisiti per il backend:** controllo di `idProperty` a ogni chiamata (S3), difesa CSRF sul cookie (S6), limite ai tentativi (S9).
- Allineamento del frontend:** gli invii con `submit()` e gli errori sotto il campo in Assistenza e nei popup (Q5, Q4), via `takeUntilDestroyed` (Q1), gestione degli errori in un solo punto (Q2). Conviene farlo ora, perché ogni nuova pagina copia il modello delle precedenti.
- Infrastruttura:** header di sicurezza anche in PROD e routing `/api` (S5), poi la CSP in Report-Only (S4).
- Manutenzione:** aggiornamento di Angular alla 22.2.x (S8), colori nel tema (Q8), test mancanti (Q9), accessibilità della tendina (Q10).